



College of Engineering and Technology
Department of Computer Science
Fundamentals of Database System

**Chapter 5 :Record Storage & Primary File
Organizations**

Storage

□ There are two general types of storage media that are used with computers. They are :

- **Primary Storage:** includes all storage media that can be operated on directly by the CPU (RAM , L1 and L2 Cache Memory)
- The first-level (L1) cache is the fastest memory in the computer and closest to the processor.
- The second-level (L2) cache is also built from SRAM but is larger, and therefore slower, than the L1 cache. The processor first looks for the data in the L1 cache.
- **Secondary Storage:** includes Hard Drives, CD's and tape.

Memory Hierarchies & Storage Devices

- The Memory hierarchy is based upon speed of access
- However, this speed comes with a price tag attached which varies inversely with the access time of memory.

Primary Storage Level of Memory

- The Primary Storage Level of Memory is generally made up of 3 Levels.
 - L1 Cache which is located on the CPU
 - L2 Cache which is located near the CPU
 - Main Memory which is RAM that is often referred to in computer advertisements

Secondary Storage Level of Memory

- The 2ndry Storage Level of Memory may be made up of 4 Levels.
 - Flash Memory or EEPROM
 - Hard Drives
 - CD ROM's
 - Tape

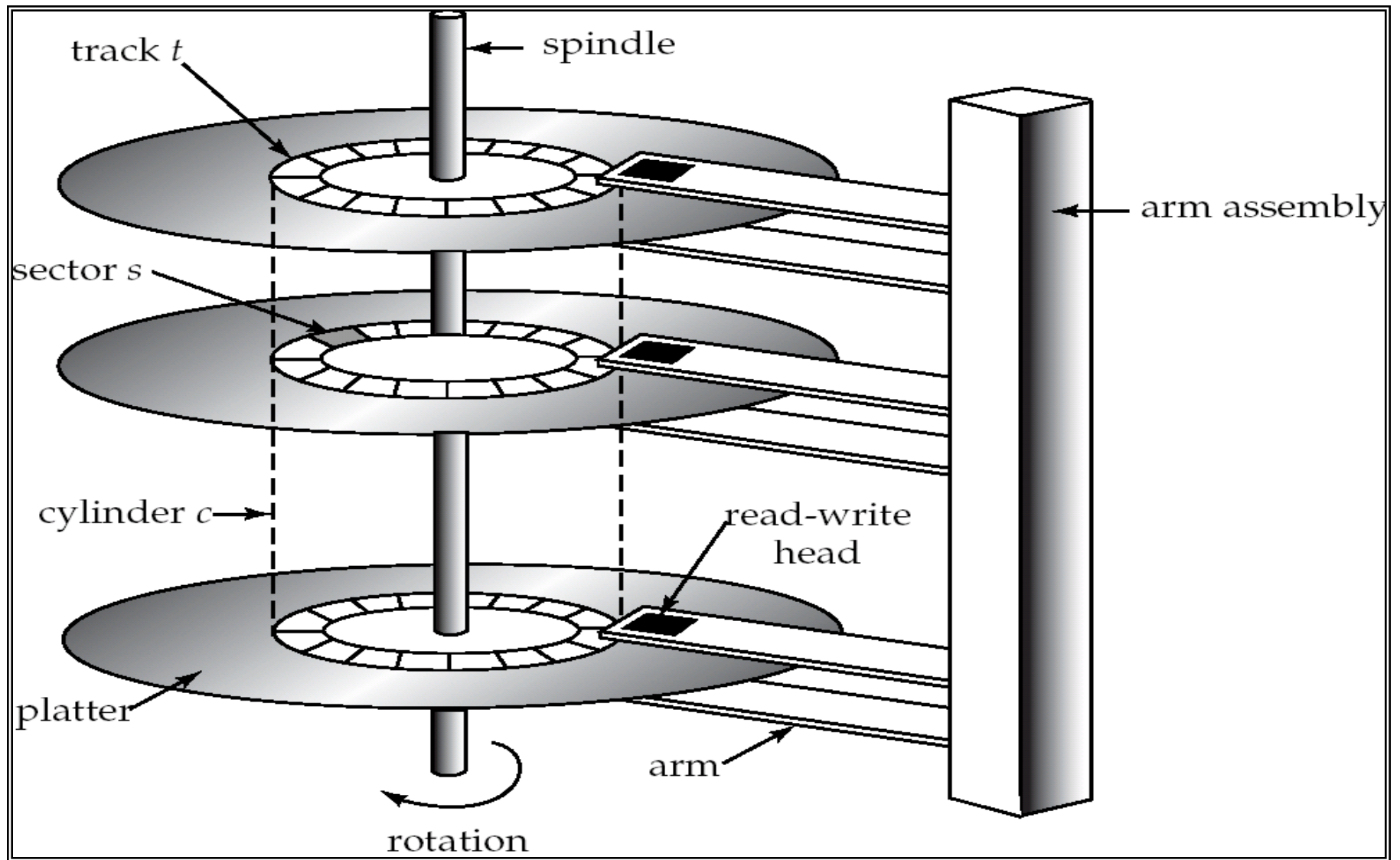


Figure 1.1

Terms Used in the HW Description of Hard Drives

- **Capacity** - The number of bytes it can store.
- **Single-sided vs. Double-sided** - States if the disk/platter is written on one or both sides.
- **Disk Pack** - A collection of disks/platters that are assembled together into a pack.
- **Track** - A Circle of a small width on a disk.
 - A disk surface will have many tracks.

Terms Used in the HW Description of Hard Drives

- **Sector** - A segment or arc of a track.
- **Block** - is the division of a track into equal sized portions by the operating system.
- **Interblock Gaps** - These are fixed sized segments that separate the blocks.
- **Read/Write Head** - Actual reads/writes the information to the disk.
- **Cylinder** - Tracks with the same diameter that are located on the disk surface of a disk pack.

Parallelizing Disk Access Using RAID

- RAID - Stands for Redundant Arrays of Inexpensive Disks or Redundant Arrays of Independent Disks.
- It is a way of logically putting multiple disks together into a single array which are working together.
- RAIDs are used to provide increased reliability, increased performance or both.

RAID Levels

- Level 0 - has no redundancy and the best write performance but its read performance is not as good as level 1.
- Level 1 - uses mirrored disks which provide redundancy and improved read performance.
- Level 2 - provides redundancy using Hamming Codes
- The Hamming Code is simply the use of extra parity bits to allow the identification of an error. Write the bit positions starting from 1 in binary form (1, 10, 11, 100, etc).

RAID Levels

- Level 3 - uses a single parity disk.
- Level 4 and 5 - use block-level data striping with level 5 distributing the data across all the disks.
- Level 6 - uses the $P + Q$ redundancy scheme making use of the Reed-Soloman codes to protect against the failure of 2 Disks.

Records

- Record is the term used to refer to a number of related values or items.
- Each value or item is stored in a field of a specific data type.
- Records may be of either fixed or variable lengths.

Variable Length Records in Files

- There are several reasons a record with the same record type may be of variable length.
 - Variable length fields
 - Repeating fields
- For efficiency reasons different record types may be clustered in a file.

Spanned Vs. Unspanned Records

- Unspanned Records: When many records are restricted to fit within one block due to their small size.
- Spanned Records: When (portions of) a single record may lie in different blocks, due to their large size.

File Operations

- File may either be stored in contiguous blocks or by linking the blocks together.
- There are advantages and disadvantages to both methods.
- Operations on files can be group into two type of operations.
Retrieval or update.
- Retrieval only involves a **read** while and update involves **read, write and modification.**

File Structure

- Heap (Pile) Files
- Ordered (Sorted) Files
- Hash (Direct) Files
- B - Trees

Heap (Pile) Files

- A heap file is an unordered set of records, stored on a set of pages.
- This class provides basic support for inserting, selecting, updating, and deleting records.
 - Insertions - Very efficient
 - Search - Very inefficient (Linear Search)
 - Deletion - Very inefficient
- Temporary heap files are used for external sorting and in other relational operators.

Ordered (Sorted Files) Records

- A sorted file is one in which records are stored in order of the values of one field (e.g., ID number) – or in order of the concatenation of several fields. (e.g., first & last names) The sort field is sometimes called as a key of the file.
- Records are stored based on the value contained in one of their fields called the *ordering field*.
- If the ordering field is also a key field than the field is better described as an *ordering key*.

Advantages of Ordered Files

- Reading of the records in order of the *ordering field* is extremely efficient.
- Finding the next record is fast.
- Finding records based on a query of the ordering field is efficient. (binary search).
- Binary search may be done on the blocks as well.

Disadvantages of Ordered Files

- Searches on non-ordering fields are inefficient.
- Insertion and deletion of records are very expensive.

Hashing Techniques

- Hashing is a technique to **directly search** the location of desired data on the disk.
- It is used to index and retrieve items in a database as it is faster to search that specific item using the shorter hashed key instead of using its original value.
- This is where a records placement is determined by value in the ***hash*** field.
- This value has a ***hash or randomizing function*** applied to it which yields the address of the disk block where the record is stored.
- For most records, we need only a single-block access to retrieve that record.

Internal Hashing

- Internal Hashing is implemented as a **hash table** through the use of an **array** of records. (In memory)
- An array index range of **0 to M-1**. A function that transforms the hash field value into an integer between **0 to M-1** is used.

Internal Hashing (con't)

- **Collisions** occur when a hash field value of a record being inserted hashes to an address that already contains a different record.
- The process of finding another position for this record is called *collision resolution*.

Collision Resolution

- **Open Addressing**- Places the record to be inserted in the *first available* position subsequent to the hash address.
- **Chaining** - A pointer field is added to each record location. When an overflow occurs this pointer is set to point to overflow blocks making a *linked list*.

Collision Resolution (con't)

- **Multiple hashing** - If an overflow occurs a *second* hash function is used to find a new location.
 - If that location is also filled either another hash function is applied or open addressing is used.

Goals of the Hash Function

- The goals of a good hash function are to ***uniformly distribute*** the records over the address space while minimizing collisions to ***avoid wasting space***.
- Research has shown
 - 70% to 90% fill ratio best.
 - That when uses a Mod function M should be a ***prime*** number.

External Hashing for Disk Files

- External hashing makes use of ***buckets***, each of which can hold ***multiple records***.
- A ***bucket*** is either a ***block or a cluster of contiguous blocks***.
- The hash function maps a key into a relative ***bucket number***, rather than an ***absolute block address*** for the bucket.

Types of External Hashing

- Using a fixed address space is called static hashing.
- Dynamically changing address space:
 - Extendible hashing*
 - Linear hashing**

Whereas,

* With a Directory

** Without a Directory

Overflow (Bucket Splitting)

- When an overflow in a bucket occurs that bucket is *split*.
- This is done by *dynamically allocating a new bucket* and *redistributing* the contents of the old bucket between the old and new buckets based on the increased local depth $d'+1$ of both these buckets.
- Where d refers to as the *global depth* of the directory.

Overflow (Bucket Splitting)

- Now the new bucket's address must be added to the directory.
- If the overflow occurred in a bucket whose current local depth d' is less than or equal to the global depth d adjust the directory entries accordingly. (No change in the directory size is made.)

Overflow (Bucket Splitting)

- If the overflow occurred in a bucket whose current local depth d' is now greater than the global depth d you must increase the global depth accordingly.
- This results in a ***doubling*** of the directory size for each time d is increased by 1 and needs appropriate adjustment of the entries.

Linear Hashing

- **Linear Hashing** allows the hash file to expand and shrink its number of buckets dynamically *without* needing a directory.
- It starts with ***M*** buckets numbered ***0 to M-1*** and use the mod hash function

$h(K) = K \bmod M$ as the initial hash function called ***h_i***.

Hashing Example with Open Addressing

Hash Function = $K \bmod M$, where K is the field value, and M is the size of the address space.

This will result in the range of values of the hash function to match the address spaces.

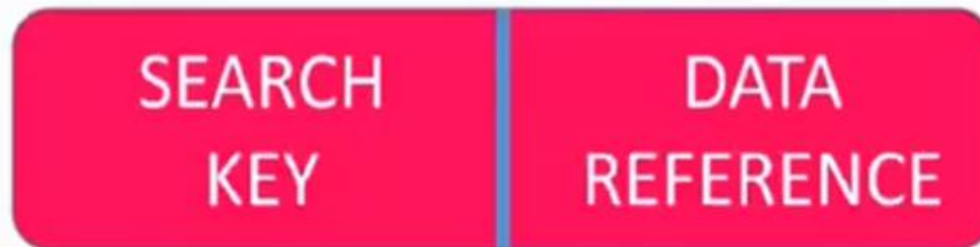
$$M = 9$$

$$h(k) = K \bmod M$$

K	$h(K)$
30	3
45	0
24	6
25	7
36	0
54	0

Indexing

- **Indexing** is **used** to **speed** up **retrieval** of records.
- It is done with the help of a **separate sequential file**.
- **Each record** of in the **index file** consists of two fields, a key field and a pointer into the main file.
- To **find** a specific **record** for the given **key value**, **index** is **searched** for the given key value.
- **Binary search** can **used** to search in **index file**. After getting the address of record from index file, the record in main file can easily be retrieved.



Index Structure for Files

❑ Types of Single level Ordered Index

- A dense index has an index entry for every search key value in the data file.
- A sparse (or non dense) index, on the other hand, has index entries for only some of the search values. A sparse index has fewer entries than the number of records in the file.

❑ Two basic kinds of indices:

- **Ordered indices:** search keys are stored in sorted order
- **Hash indices:** search keys are distributed uniformly across “buckets” using a “hash function”.

Reading Assignment

Dynamic Multilevel indexes using B Trees and B+ Trees, Multiple Indexes

End of Chapter 5

Thank you!